

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

Work Integrated Learning Programmes Division

Cluster Programme - M. Tech in AI & ML

II Semester , 2023 – 24(October,2024)

COMPREHENSIVE Examination (MAKEUP)_ANSWER KEY

Course No : AIMLC ZG511
Course Title : DEEP NEURAL NETWORK
Nature of Exam. : Open Book
Weightage : 35 Marks
Duration : 120 minutes
Date : 6th October ,2024_2 PM

Number of questions:4 Number of Pages: 2

Q.1. The following code snippet is used for a classification problem. **[10 Marks]**

```
dnnModel = models . Sequential ( )  
dnnModel .add ( layers . Dense (60 , activation=' tanh ' , input_shape= (64_64 , ) ) )  
dnnModel .add ( layers . Dense (40 , activation=' relu ' ) )  
dnnModel .add ( layers . Dense (20 , activation = ' relu ' ) )  
dnnModel .add ( layers . Dense (10 , activation=' relu ' ) )  
dnnModel .add ( layers . Dense ( 4 , activation=' softmax ' ) )
```

a. Find the number of hidden layers. **[1 Mark]**

ANSWER: 4

b. Find the number of categories that are being classified. **[1 Mark]**

ANSWER: 4

c. Compute the size of input vector. **[1 Mark]**

ANSWER: $64 * 64 = 4096$

d. Draw the network given by the code snippet. **[1 Mark]**

ANSWER:

e. When will you use a sigmoid against a softmax activation function in the output layer of a DNN. **[2 Marks]**

ANSWER: Sigmoid function is used to represent a probability distribution over a binary Variable. Use the softmax function to represent a probability distribution over a discrete variable with n possible values.

f. Compare tanh and ReLU activation functions in terms of gradients. **[2 Marks]**

ANSWER: equations ($e^z - e^{-z}$)

($e^z + e^{-z}$) and $\max(0, z)$

tanh is differentiable at all points. relu is undefined at 0.

tanh lead to vanishing gradient. ReLU lead to exploding gradient.

g. Compare momentum based gradient descent algorithm against RMSprop. **[2 Marks]**

ANSWER: RMSProp adjusts learning rates based on recent gradients' magnitude, while momentum accelerates convergence by accumulating past gradients' direction and velocity.

Q.2. a). Consider the following code snippet and draw the corresponding architecture. And also calculate number of parameters. **[4 Marks]**

```
Model = Sequential()
```

```
Model.add(SimpleRNN(4, input_shape=(4,6),activation=activation[0]))
```

```
Model.add(Dense(1,activation="Sigmoid"))
```

ANSWER:

Architecture Diagram: 2 Marks

Total number of Parameters:

RNN Layer: $W_{xh} + W_{hh} + b_h$

$$= 4*6 + 4*4 + 4 = 44$$

[1/2 M]

Output Layer: $W_o = 1*4$ and $b_o: 1 \quad 1*4 + 1 = 5$

[1/2 M]

Ttal number of parameters: $44+5 = 49$

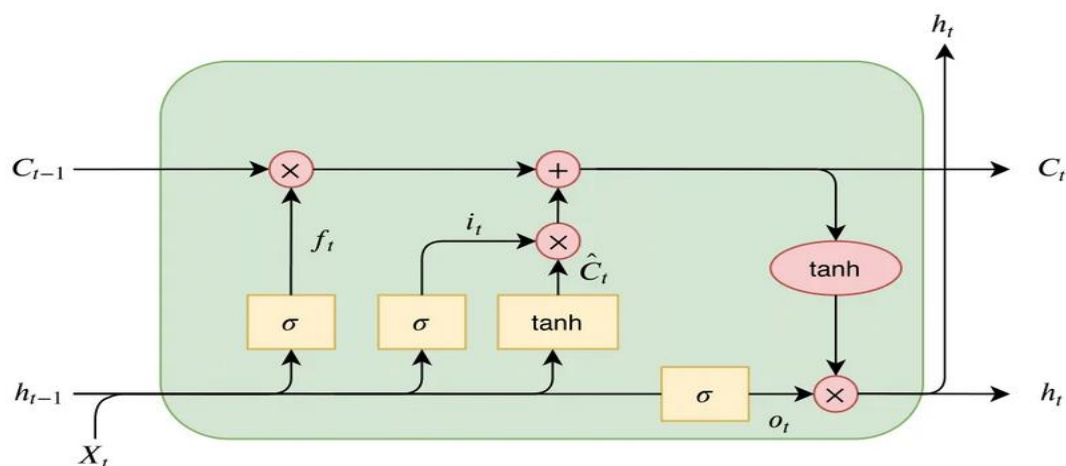
[1 M]

b). Consider the below architecture.

i) If X_t, h_t, C_t are of the dimensions 4, 5 and 5 respectively, then find the number of parameters to be trained at t . Show the calculations / justifications. **[4 Marks]**

ii) Justify the following statement. **[2 Marks]**

"In the above architecture ' σ ' can be replaced with ' $\tanh$ ' to obtain the optimal architecture"



ANSWER

i) Given Dimensions of X_t, h_t and C_t are 4, 5 and 5 resp

ft gate corresponds to forget gate with inputs X_t and h_t adn outputs dimensions of size C_t thus size of weight matrix $W_f = 5 * [4+5] = 45$

Award 1 mark for figuring out that output dimension is same as c_t

Award 1 mark for figuring out $5*9 = 45$

bias bf: 5 [1 M]
total number of parameters: 45+5 = 50 [1 M]
Deduct half mark if total number of parameters i.e. 50 is not written

ii). In the given architecture, activations for forget gate; input to input gate and input the output gate all have sigmoid activations.
Of the three, replacing sigmoid activation with tanh to get it will result in better architecture because it provides more control over selective read i.e. how much information do we read and carry forward. [2 M]

Q.3. Given a set of query vectors $[Q_1, Q_2]$, key vectors $[K_1, K_2]$ and value vectors $[V_1, V_2]$ where $Q_1 = [1, 0]$, $Q_2 = [0, 1]$, $K_1 = [1, 1]$, $K_2 = [0, 1]$, $V_1 = [2, 1]$, $V_2 = [0, 3]$, calculate the attention scores and attention-weighted outputs for both queries using dot-product attention and softmax. [8 Marks]

Solution:

Step 1: Compute the Attention Scores 4 Marks
The attention score is computed by taking the dot product between the query vectors and the key vectors.

1. Attention score for Q_1 with K_1 :
[
. {Score} $Q_1, K_1 = Q_1 \cdot K_1 = [1, 0] \cdot [1, 1] = 1 * 1 + 0 * 1 = 1$
]

2. Attention score for Q_1 with K_2 :
[
. {Score} $Q_1, K_2 = Q_1 \cdot K_2 = [1, 0] \cdot [0, 1] = 1 * 0 + 0 * 1 = 0$
]

3. Attention score for Q_2 with K_1 :
[
. {Score} $Q_2, K_1 = Q_2 \cdot K_1 = [0, 1] \cdot [1, 1] = 0 * 1 + 1 * 1 = 1$
]

4. Attention score for Q_2 with K_2 :
[
. {Score} $Q_2, K_2 = Q_2 \cdot K_2 = [0, 1] \cdot [0, 1] = 0 * 0 + 1 * 1 = 1$
]

So the attention scores are:

	K1	K2	
Q1	1	0	

Q2	1	1	
----	---	---	--

Mark Distribution:

- 1 mark for calculating Score_{Q_1, K_1}
- 1 mark for calculating Score_{Q_1, K_2}
- 1 mark for calculating Score_{Q_2, K_1}
- 1 mark for calculating Score_{Q_2, K_2}

Step 2: Apply Softmax to the Attention Scores 2 Marks

Next, apply the softmax function to normalize the attention scores for each query. The softmax function is calculated as follows:

$$[\text{Softmax}]_{x_i} = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

1. Softmax for Q_1 :

$$[\text{Softmax}]_{Q_1} = \left[\frac{e^1}{e^1 + e^0}, \frac{e^0}{e^1 + e^0} \right] = \left[\frac{e^1}{e^1 + 1}, \frac{1}{e^1 + 1} \right] = \left[\frac{e}{e + 1}, \frac{1}{e + 1} \right]$$

2. Softmax for Q_2 :

$$[\text{Softmax}]_{Q_2} = \left[\frac{e^1}{e^1 + e^1}, \frac{e^1}{e^1 + e^1} \right] = \left[\frac{1}{2}, \frac{1}{2} \right]$$

Softmax outputs:

	K1	K2	
Q1	$\frac{e}{e+1}$	$\frac{1}{e+1}$	
Q2	0.5	0.5	

Mark Distribution:

- 1 mark for applying softmax to Q_1
- 1 mark for applying softmax to Q_2

Step 3: Calculate Attention-Weighted Output 2 Marks

Finally, the attention-weighted output is calculated by taking a weighted sum of the value vectors, weighted by the softmax scores.

1. Attention-weighted output for Q_1 :

$$[\text{Output}]_{Q_1} = \left[\frac{e}{e+1} \right] \cdot V_1 + \left[\frac{1}{e+1} \right] \cdot V_2$$

$$\begin{aligned}
 & \left[\begin{aligned} & \text{Output}_{Q_1} = \begin{bmatrix} \{e\} \{e+1\} \end{bmatrix} \cdot \begin{bmatrix} 2, 1 \end{bmatrix} + \begin{bmatrix} \{1\} \{e+1\} \end{bmatrix} \cdot \begin{bmatrix} 0, 3 \end{bmatrix} \\ & \end{aligned} \right] \\
 & \left[\begin{aligned} & \text{Output}_{Q_1} = \begin{bmatrix} \{2e\} \{e+1\}, \{e\} \{e+1\} \end{bmatrix} + \begin{bmatrix} 0, \{3\} \{e+1\} \end{bmatrix} = \begin{bmatrix} \{2e\} \{e+1\}, \{e+3\} \{e+1\} \end{bmatrix} \\ & \end{aligned} \right]
 \end{aligned}$$

2. Attention-weighted output for Q_2 :

$$\begin{aligned}
 & \left[\begin{aligned} & \text{Output}_{Q_2} = 0.5 \cdot V_1 + 0.5 \cdot V_2 \\ & \end{aligned} \right] \\
 & \left[\begin{aligned} & \text{Output}_{Q_2} = 0.5 \cdot \begin{bmatrix} 2, 1 \end{bmatrix} + 0.5 \cdot \begin{bmatrix} 0, 3 \end{bmatrix} = \begin{bmatrix} 1, 0.5 \end{bmatrix} + \begin{bmatrix} 0, 1.5 \end{bmatrix} = \begin{bmatrix} 1, 2 \end{bmatrix} \\ & \end{aligned} \right]
 \end{aligned}$$

Final Outputs:

	Output	
Q1	$\frac{\{2e\}}{\{e+1\}}, \frac{\{e+3\}}{\{e+1\}}$	
Q2	0.5	

	Output	
Q_1	$\left[\frac{\{2e\}}{\{e+1\}}, \frac{\{e+3\}}{\{e+1\}} \right]$	
Q_2	$[1, 2]$	

Mark Distribution:

- 1 mark for calculating the weighted output for Q_1
- 1 mark for calculating the weighted output for Q_2

Total Marks Distribution:

- 4 marks for attention score calculations
- 2 marks for applying softmax
- 2 marks for attention-weighted output

Total: 8 Marks

Q.4.

a. Briefly explain how the method One-Shot Architecture Search works. **[2 Marks]**

Answer: (0.5 marks for each point)

- Define a "supergraph" that encompasses a wide range of potential architectures. Each individual architecture is represented as a smaller, connected subgraph within this supergraph.

- Instead of training each candidate architecture individually, **only the one-shot model (the supergraph) is trained**.
 - To evaluate a specific architecture, **activate the corresponding subgraph** within the trained one-shot model.
 - **Weights for the active connections are inherited** from the trained one-shot supergraph.
- b. Discuss ONE potential advantage and ONE potential disadvantage of using this method for performance estimation in NAS. **[2 Marks]**

Answer:

Advantages: (any one is sufficient – 1 mark)

- **Better Efficiency** - Only requires training a single, over-parameterized supergraph, drastically reducing the computational cost compared to training numerous individual architectures.
- **Fast Evaluation** - Evaluating new architectures is fast as it only involves inheriting trained weights and requires no separate training.

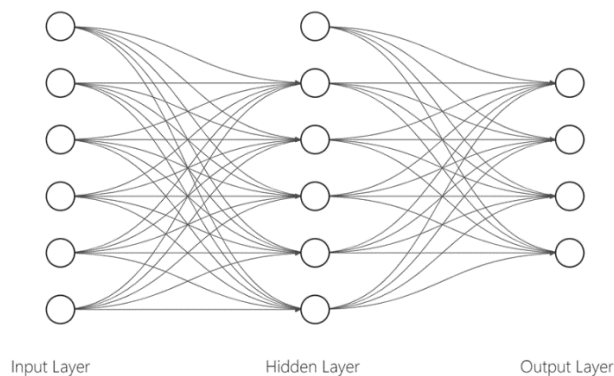
Disadvantages: (1 mark)

- **Bias Towards Supergraph** - Performance estimates are heavily dependent on the design and training quality of the supergraph. A poorly designed or trained supergraph can mislead the search and limit the discovery of optimal architectures.

- c. Consider a fully connected 3 layered feed forward neural network in which all the layers (except output layer) contain 5 neurons and each output layer contains 4 neurons. Draw the architecture of the above mentioned. And show the total number of learnable parameters at each layer and also of the network. **[3 Marks]**

Answer: (1 mark for architecture and 2 marks for learnable parameters)

Architecture:



Parameters Per Layer:

Input Layer: 0 learnable parameters (no weights or biases)

Hidden Layer:

- Weights: 25 (5 input layer neurons * 5 hidden layer neurons)
- Biases: 5
- Total: 30 learnable parameters

Output Layer:

- Weights: 20 (5 hidden layer neurons * 4 output layer neurons)
- Biases: 4
- Total: 24 learnable parameters

Total Network learnable Parameters: $0 + 30 + 24 = 54$ learnable parameters.

The full marks shall be given if any student considered 2 hidden layers (instead of one hidden layer):

Architecture: Add one hidden layer to the above architecture.

Parameters Per Layer:

Input Layer: 0 learnable parameters (no weights or biases)

Hidden Layer - 1:

- Weights: 25 (5 input layer neurons * 5 hidden layer neurons)
- Biases: 5
- Total: 30 learnable parameters

Hidden Layer - 2:

- Weights: 25 (5 input layer neurons * 5 hidden layer neurons)
- Biases: 5
- Total: 30 learnable parameters

Output Layer:

- Weights: 20 (5 hidden layer neurons * 4 output layer neurons)
- Biases: 4
- Total: 24 learnable parameters

Total Network learnable Parameters: $0 + 30 + 30 + 24 = 84$ learnable parameters.

XXXXX